Milestone 3 |E01

Retail Revenue Preparation

SUBMISSION TEMPLATE

Difficulty	Marks	Tasks	Working budget
Easy	10	4	25 minutes

Trainee: _____Soumiya Ayyappan_____ Candidate ID: ___10877547_____ Catalog: _____

Objective

Prepare paid retail order revenue with a managed Delta detail and reconcile regional amounts.

Evidence policy

There is NO fixed one-task/one-screenshot limit. Attach clear and readable images for every assessed criterion. If code AND results are required, show BOTH: separate Code_A/Code_B and Result_A/Result_B slots may be used. When code does not fit, capture its consecutive portions in two or three screenshots; never shrink or stitch screenshots until unreadable. For UI-based tasks, show the exact configuration and saved output. Preserve outputs: do not repeat MERGE/UPDATE/DELETE or restart pipelines merely to photograph them. The evaluator scores only submitted visible evidence. A workspace URL is useful for reference, but does not replace evidence.

Task requirements and evidence holders

Task 1. Prepare governed source | 2 marks

1.1 Use your assigned catalog and milestone3 schema. Create the question EXTERNAL volume under the instructor-supplied ADLS child URI and upload the exact CSV in input/.

1.2 Read six columns with the declared types; verify the number of input records.

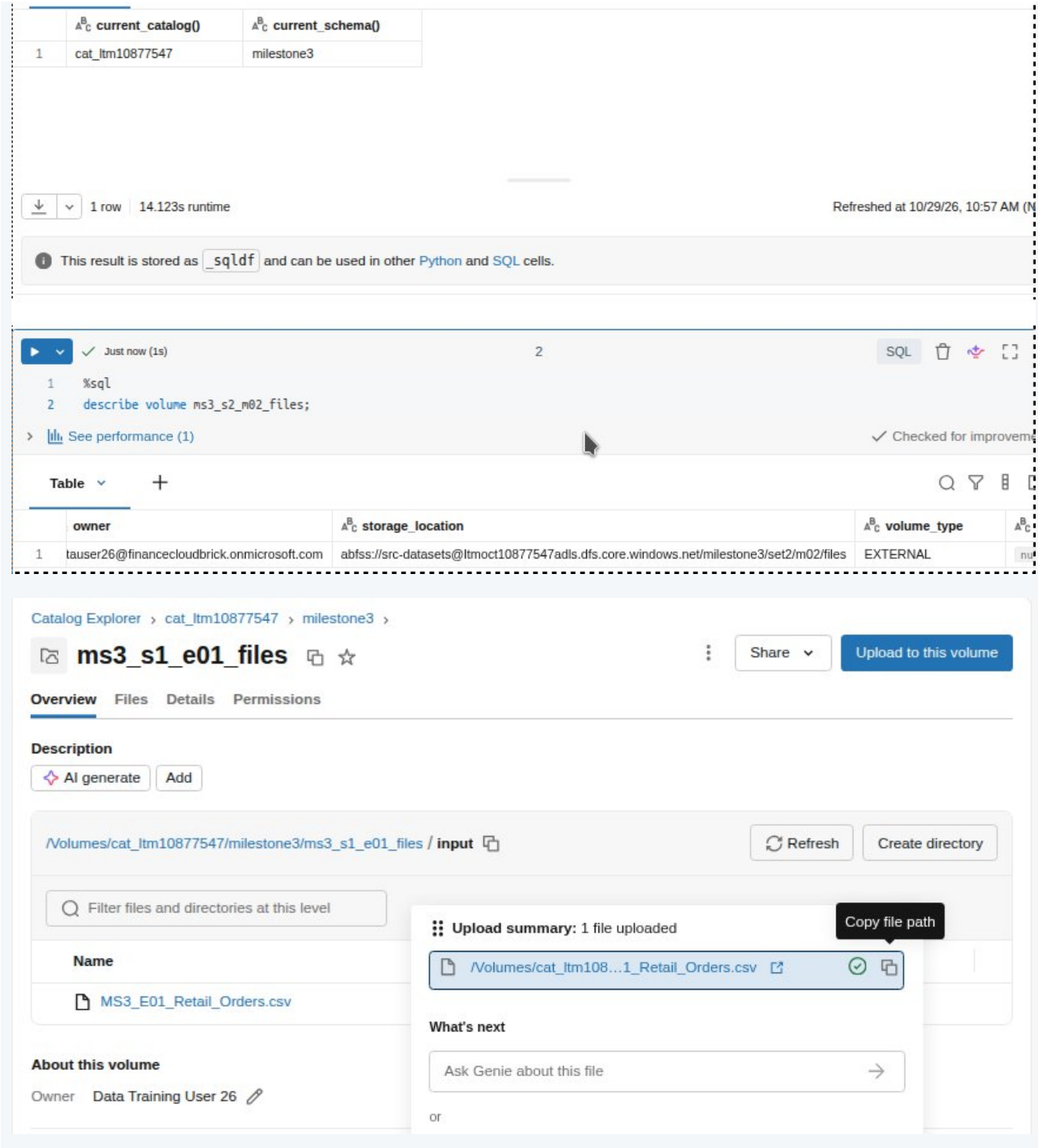
Expected output

Eight source rows and six columns.

Required screenshots (code / result / UI)

A. [UI] EXTERNAL volume name, full URI and exact input filename. Suggested name: MS3_S1_E01_T01_UI_A.png. Additional numbered screenshots are allowed when necessary.

PASTE UI SCREENSHOT A BELOW:



B. [Code] Source path and declared typed read. Suggested name: MS3_S1_E01_T01_Code_B.png. Additional numbered screenshots are allowed when necessary.

PASTE CODE SCREENSHOT B BELOW:

```python
from pyspark.sql.functions import *
from pyspark.sql.types import *
schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("region", StringType(), True),
    StructField("category",StringType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("unit_price", DecimalType(12,2), True),
    StructField("status", StringType(), True)
])
path="/Volumes/cat_ltm10877547/milestone3/ms3_s1_e01_files/input/MS3_E01_Retail_Orders.csv"
df=spark.read.csv(path,header=True,schema=schema)
display(df.count())
```

C. [Result] Source count 8 and listed columns. Suggested name: MS3_S1_E01_T01_Result_C.png. Additional numbered screenshots are allowed when necessary.

PASTE RESULT SCREENSHOT C BELOW:




```python
display(df)
```

See performance (1) ✓ Checked for improvements

Table +

	order_id	region	category	quantity	unit_price
1	R101	North	Electronics	2	200.00
2	R102	South	Clothing	3	50.00
3	R103	North	Grocery	4	25.00
4	R104	West	Electronics	1	300.00
5	R105	South	Clothing	0	80.00
6	R106	West	Grocery	2	-10.00
7	R107	West	Electronics	1	250.00
8	R108	North	Clothing	2	60.00

Extra screenshot(s) for this task: _____ Workspace reference: _____

Task 2. Filter eligible orders | 3 marks

2.1 Keep rows with PAID status, quantity > 0 and unit_price > 0.

2.2 List included order IDs and excluded IDs with reason.

Expected output

Keep R101,R102,R103,R107,R108 (5); exclude R104 pending, R105 zero quantity, R106 negative price.

Required screenshots (code / result / UI)

A. [Code] All three eligibility conditions. Suggested name: MS3_S1_E01_T02_Code_A.png. Additional numbered screenshots are allowed when necessary.

PASTE CODE SCREENSHOT A BELOW:

```
from pyspark.sql.functions import when, col

clean_df = df.filter((col("status") == "PAID") & (col("quantity") > 0) & (col("unit_price") > 0))

excluded_df = df.subtract(clean_df).withColumn(
    "exclusion_reason",
    when(col("status") != "PAID", concat(lit("Non-PAID status ("), col("status"), lit(")")))
    .when(col("quantity") <= 0, "Zero or negative quantity")
    .when(col("unit_price") <= 0, "Zero or negative price")
    .otherwise("Unknown")
)

print(f"Included rows ({clean_df.count()}):")
display(clean_df.select("order_id"))
print(f"Excluded rows ({excluded_df.count()}):")
display(excluded_df.select("order_id", "exclusion_reason"))
```

B. [Result] Five retained IDs and three excluded IDs/reasons. Suggested name: MS3_S1_E01_T02_Result_B.png. Additional numbered screenshots are allowed when necessary.

PASTE RESULT SCREENSHOT B BELOW:

Included rows (5):

Table

	order_id
1	R101
2	R102
3	R103
4	R107
5	R108

5 rows | 2.007s runtime

Excluded rows (3):

Table

	order_id	exclusion_reason
1	R104	Non-PAID status (PENDING)
2	R106	Zero or negative price
3	R105	Zero or negative quantity

Extra screenshot(s) for this task: _____ Workspace reference: _____

Task 3. Calculate and save revenue | 3 marks

3.1 Calculate line_revenue and store to two decimal places.

3.2 Save six original columns plus line_revenue as managed Delta table ms3_s1_e01_retail_sales.

3.3 Show order_id and line_revenue sorted by order_id.

Expected output

R101 400.00; R102 150.00; R103 100.00; R107 250.00; R108 120.00. Five saved rows.

Required screenshots (code / result / UI)

A. [Code] Calculation and named managed Delta persistence. Suggested name: MS3_S1_E01_T03_Code_A.png. Additional numbered screenshots are allowed when necessary.

PASTE CODE SCREENSHOT A BELOW:

```
revenue_df = clean_df.withColumn("line_revenue", round(col("quantity") * col("unit_price"), 2).cast(DecimalType(12, 2)))

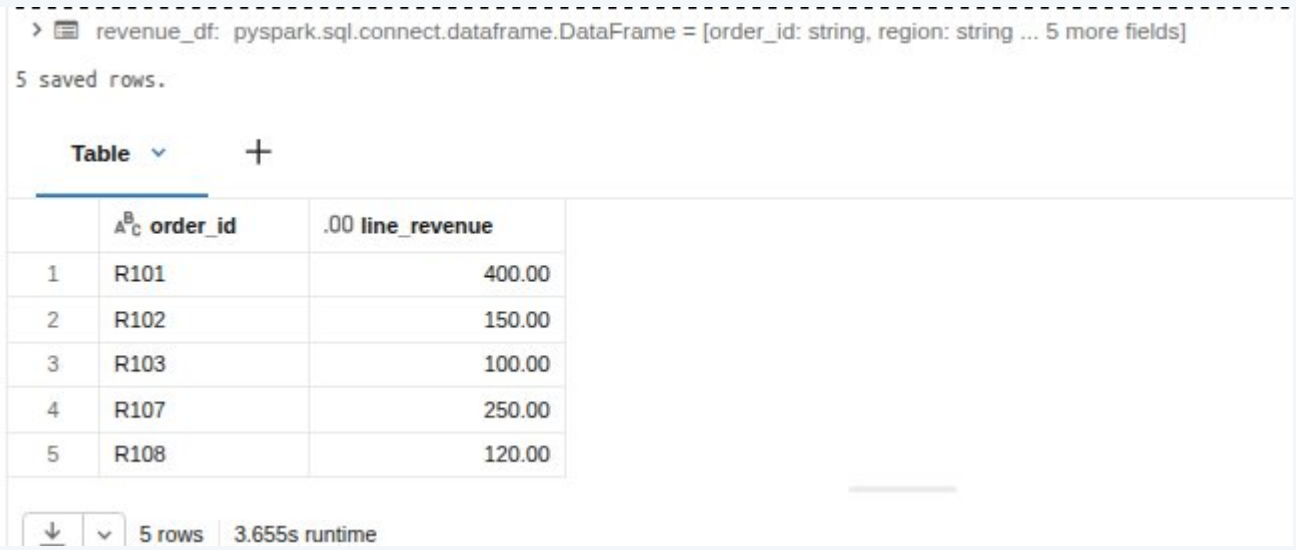

revenue_df.select("order_id", "region", "category", "quantity", "unit_price", "status", "line_revenue") \
    .write.format("delta").mode("overwrite").saveAsTable("ms3_s1_e01_retail_sales")

print(f"{revenue_df.count()} saved rows.")


display(revenue_df.select("order_id", "line_revenue").orderBy("order_id"))
```

B. [Result] Five saved order IDs and values. Suggested name: MS3_S1_E01_T03_Result_B.png. Additional numbered screenshots are allowed when necessary.

PASTE RESULT SCREENSHOT B BELOW:



	order_id	line_revenue
1	R101	400.00
2	R102	150.00
3	R103	100.00
4	R107	250.00
5	R108	120.00

5 rows | 3.655s runtime

Extra screenshot(s) for this task: _____ Workspace reference: _____/Workspace/Users/datauser26@financecloudbrick.onmicrosoft.com/finaleasy1_____

Task 4. Reconcile regional revenue | 2 marks

4.1 From saved Delta table, group by region and total line_revenue.

4.2 Show each region and the overall revenue sum.

Expected output

North 620.00; South 150.00; West 250.00; grand total 1020.00.

Required screenshots (code / result / UI)

A. [Code] Region grouping and SUM from saved table. Suggested name: MS3_S1_E01_T04_Code_A.png. Additional numbered screenshots are allowed when necessary.

PASTE CODE SCREENSHOT A BELOW:

```
spark.sql("select region, sum(line_revenue) from ms3_s1_e01_retail_sales group by region").show()
```

See performance (1)

```
%sql
select sum(line_revenue) from ms3_s1_e01_retail_sales;
```

B. [Result] Three region totals and 1020.00 overall. Suggested name: MS3_S1_E01_T04_Result_B.png. Additional numbered screenshots are allowed when necessary.

PASTE RESULT SCREENSHOT B BELOW:

```
+------+-----------------+
|region|sum(line_revenue)|
+------+-----------------+
| North|           620.00|
| South|           150.00|
|  West|           250.00|
+------+-----------------+
```

Table +

	.00 sum(line_revenue)
1	1020.00

Extra screenshot(s) for this task: _____ Workspace reference: _____/Workspace/Users/datauser26@financecloudbrick.onmicrosoft.com/finaleasy1_____

Expected final results

Task 1: Eight source rows and six columns.

Task 2: Keep R101,R102,R103,R107,R108 (5); exclude R104 pending, R105 zero quantity, R106 negative price.

Task 3: R101 400.00; R102 150.00; R103 100.00; R107 250.00; R108 120.00. Five saved rows.

Task 4: North 620.00; South 150.00; West 250.00; grand total 1020.00.

Evaluation rubric / evidence map

Task	Criterion	Marks	Evidence
1	Correct volume, URI, input file	1	UI
1	Typed read and input count	1	Code+Result

2	All three filter conditions	2	Code
2	Correct retained/rejected records	1	Result
3	Correct revenue formula and persistence	2	Code
3	Correct saved values	1	Result
4	Correct saved-table aggregate	1	Code
4	Correct all totals	1	Result

Submission requirements

Submit this question's matching template with readable taskwise screenshots. Screenshots may exceed the suggested slot count; both code and output must be present when rubric requires both. Keep saved objects and version history available; use the actual version identifiers. No executable solution code or validation scripts are supplied in this document. There are no hidden rubric criteria.